There is no complete guarantee that an AI coding agent will always follow natural-language guidelines, skills, or specs. Agents are probabilistic generators; the same specification can produce different implementations, dependency choices, failure modes, or security properties. Natural-language instructions are ambiguous, agents rarely proactively retrieve and adhere to external rules (one study found agents opened relevant policy files in only ~3.5% of unaided runs), and compliance recovers mainly for additive rules (disclosure/verification) but fails for restraint rules (refusal/handoff). Security studies have repeatedly found high vulnerability rates in unconstrained AI-generated code (e.g., Veracode reports around 45% overall failure rates in tested samples, higher for some languages).

The practical consensus is layered defense: make intent executable and checkable, enforce mechanically where possible, monitor and recover at runtime, quantify residual risk probabilistically, and keep humans in high-stakes loops. Pure “spec-as-source” (code is disposable and regenerated solely from the spec) carries the highest risk because of nondeterminism and incomplete capture of unknowns; “spec-first” (guide one implementation then step aside) or “spec-anchored” (maintain the spec as a living reference) are lower-risk.

Structured Spec-Driven Processes

Tools such as GitHub Spec Kit (and similar systems like Amazon Kiro or OpenSpec) formalize a workflow that keeps the specification as the primary artifact rather than a one-off prompt.

Typical phases (Spec Kit example):

- Constitution (project-wide principles, quality standards, testing rules).
- Specify (what and why, user stories, acceptance criteria, non-goals).
- Clarify / Plan (technical approach, constraints).
- Tasks (atomic, ordered work items).
- Implement.
- Analyze / Converge (consistency checks and validation until the implementation matches the spec).

Commands are usually agent skills (e.g., `/speckit-specify`, `/speckit-plan`, `/speckit-converge`). Human review is expected between phases. Specs live in the repository as versioned files so intent survives chat sessions. Empirical work on real SDD practice shows practitioners report more reliable agent behavior but also higher planning/review costs, selective use (mainly for new features), and ongoing risks of spec–code drift.

Addy Osmani and others emphasize modular tasks, explicit self-checks, and treating the spec as a living document. Good specs use checkable acceptance criteria (e.g., “an empty list shows the invite button” or “ `pnpm test auth/session.test.ts` passes”), out-of-scope boundaries, and constraints. Vague criteria (“handles errors well”) fail.

Making Guidelines Executable (Tests, Linters, Architectural Checks)

Natural language is weak; executable contracts are stronger.

- **Test-driven development (TDD) as specification.** Write (or tightly control) failing tests first from the acceptance criteria. Confirm they fail for the expected reason, treat the tests as largely read-only during implementation, then have the agent make them pass. Agents tend to write implementation then tautological tests that pass by construction, or even edit/skip tests; protecting the test suite is essential. Multiple guides emphasize Red → Green → Refactor with phase-specific prompts and human ownership of the test list (the real specification).
- **Custom linters and AST-based architectural tests.** Encode conventions as ESLint rules (`no-restricted-syntax` , `no-restricted-imports` , custom rules) or Python AST checks that fail on violations (e.g., controllers must not import repositories directly, no arithmetic on monetary `.amount` fields). Run them after agent edits via hooks (Claude Code PostToolUse, etc.) so the agent receives deterministic feedback and can self-heal. CI runs the same checks. This turns “prefer functional components” or “use the existing Result type” into something the agent cannot silently ignore. Examples include eslint-plugin-ai-guardrails, framework-specific plugins (e.g., Medusa), and custom rules for domain invariants.
- **Property-based testing and formal methods.** Where requirements can be stated as properties, generate or use property-based tests. More advanced work uses verification languages (Verus via PDD Kit for proof-driven development, Dafny, etc.) so the agent produces code plus proofs that the implementation meets a formal spec for all inputs. This is stronger than testing but requires more up-front specification skill and is still maturing for everyday web apps.

Spec-to-code compliance skills (e.g., Trail of Bits-style) systematically compare documentation to implementation and classify gaps (implemented, partial, contradicted, absent, stronger/weaker).

Runtime Contracts, Monitoring, and Recovery

For agent behavior itself (tool use, long-running sessions, multi-agent pipelines):

- **Agent Behavioral Contracts (ABC) and AgentAssert.** A formal Design-by-Contract style framework defines a contract $\mathcal{C} = (\mathcal{P}, \mathcal{I}, \mathcal{G}, \mathcal{R})$ covering preconditions, hard/soft invariants, governance policies, and recovery. Hard constraints are zero-tolerance; soft ones allow recovery within a window k. It defines probabilistic (p, δ, k)-satisfaction and a Drift Bounds Theorem: with recovery rate $\gamma >$ natural drift rate α, expected drift is bounded by $D^* = \alpha/\gamma$ (with Gaussian concentration). Implementation (YAML contracts + Python runtime library) detects soft violations that uncontracted agents miss, achieves high hard-compliance rates (reported 88–100% in evaluations), bounds drift, and adds <10 ms overhead. Recovery turns exponential compliance decay into roughly linear decay. This is a concrete way to quantify and bound residual risk rather than hoping the prompt sticks.
- Related approaches include compiling company policies into tool-use guards (ToolGuards), runtime monitors, and agentic behavior engineering that treats observable behavior as the versioned artifact.

Multi-agent setups help: one agent implements, another reviews for security/architecture/spec conformance, with explicit quality gates. Human checkpoints remain necessary for irreversible or high-impact decisions.

Quantifying Risks

- **Empirical failure rates.** Security benchmarks, compliance studies (low unaided rule following), and SDD practice research (drift types: interpretation, edit, regeneration, runtime).
- **Metrics.** Pass^k (reliability under repeated trials), compliance scores, behavioral drift scores (compliance gap + distributional shift), recovery effectiveness, and stress resilience under adversarial conditions.
- **Process signals.** Spec–code divergence frequency, number of clarifying questions an agent asks, divergence between models interpreting the same spec, mutation scores of tests.

Studies note that more detailed specs do not always improve architectural quality and that review of artifacts can become the new bottleneck. Governance at scale (versioning specs, ownership, linking spec versions to deployed behavior) is still immature.

Practical Handling Strategy

1. Prefer spec-first or spec-anchored over pure regeneration-from-spec for anything long-lived or high-stakes.

2. Write checkable acceptance criteria and turn as many guidelines as possible into tests, linters, or runtime contracts.
3. Use structured workflows (Spec Kit or equivalent) with human gates on plans and high-risk changes.
4. Enforce mechanically (CI, post-edit hooks, tool guards) so the agent receives deterministic feedback and can self-correct.
5. Add runtime monitoring and recovery for agent behavior; quantify residual risk with pass ^{k}, drift bounds, or contract satisfaction levels.
6. Keep independent verification (second agent or human) for security, architecture, and non-functionals. Link everything for auditability.
7. Accept residual risk: unknowns, non-determinism, and context limits cannot be fully eliminated. Automate what is checkable; reserve human judgment for what is not.

Key resources

- GitHub Spec Kit: github.com/github/spec-kit (quickstart and command references available).
- AgentAssert / ABC papers and library (agentassert.com, arXiv:2602.22302, associated GitHub).
- Practical articles on executable guardrails (Nearform), TDD with agents, and SDD limits (UngerAI analysis of risk levels and 18 limitations).
- Compliance and failure-mode papers on coding-agent rule adherence and ungoverned generation risks.

This combination—structured specs + executable checks + runtime contracts + quantified residual risk + selective human oversight—is the current practical answer. It does not eliminate unexpected behavior, but it makes deviations detectable, recoverable, and measurable so automation can be used with known bounds rather than hope.